Introduction (Mocking vs Stubbing vs Spy)

Mocking:

In Unit test cases, we test a unit in isolation.

And a unit could be:

- Single public method of a class
- Or Single behavior or responsibility.

But in real world classes do not live alone, it depends on:

- Other classes
- External API
- DB
- Message Broker
- File system

Etc..

Concept && Coding

Mocking helps achieve isolation by replacing real dependencies with fake (Mock) objects, so that our unit test do not use the real object.

```
class OrderService {
    private PaymentService paymentService;

    OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    boolean placeOrder(int amount) {

           if(paymentService.pay(amount)) {
            return true;
           }
           else {
                 return false;
           }
    }
}

```

```
class PaymentService {

    boolean pay(int amount) {
              BankResponse response =   //calls real bank server
               return response.isPaymentSuccess();
        }
}

```

OrderService "*placeOrder(int amt)*" invokes PaymentService "*pay(int amt)*" for placing the order.

So, when we write Unit test cases for OrderService "*placeOrder*" method, we should not invoke PaymentService or any other class, we need to MOCK (fake object) them.

For example:

Test case scenario 1: Place an order and payment transaction get success.

```java
@Test
void placeOrderSuccess() {

   //Fake object(Mock) of PaymentService Class.
  PaymentService paymentService = mock(PaymentService.class);
  //Mock object is passed to SUT (system under test).
  OrderService orderService = new OrderService(paymentService);
  /*
  Using Mock object, I can define what value should be returned when a
  particular method is invoked.
  */
    when(paymentService.pay(10)).thenReturn(true);

    boolean result = orderService.placeOrder(10);
  /*
    Here we are testing the OrderService "placeOrder" method outcome,
    when PaymentService "pay" method is succeeded , "placeOrder" should also
    return true.
  */
    assertTrue(result);
}

```

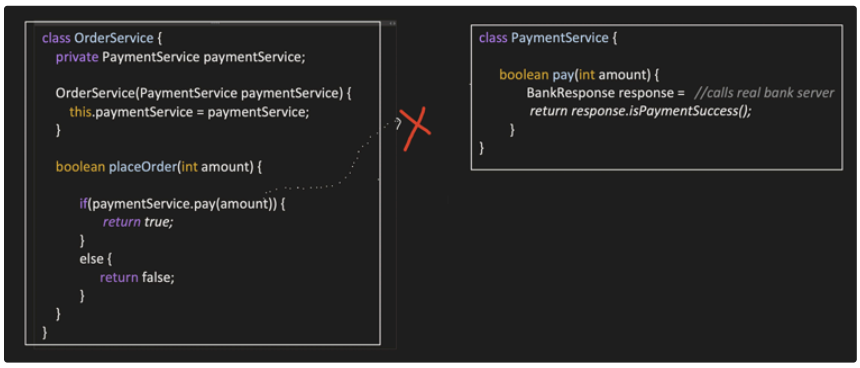
Concept && Coding

So above, we mocked the call *paymentService.pay(10)*.
Mocking means: instead of invoking the real method, we tell the test framework, "when this method is called, just return the predefined value without executing the actual logic."

Test case scenario 2: Place an order and payment transaction got failed.

```java
@Test
void placeOrderFailure() {

    PaymentService paymentService = mock(PaymentService.class);

   OrderService orderService = new OrderService(paymentService);
  /*
  Since we need to test, how "placeOrder" method behaves when Payment got failed.
  That’s why we are returning false when pay method on Mock object get invoked.
  */
    when(paymentService.pay(10)).thenReturn(false);

    boolean result = orderService.placeOrder(10);
  /*
  So expectation is, when payment get failed, "placeOrder" should also return false.
  */
    assertFalse(result);
}

```

So now, in both the above scenarios, OrderService "*placeOrder*" method do not invoke PaymentService "*pay*" method



<u>What Mocking solves?</u>

1. **Isolation:**
   As discussed in above 2 scenarios itself. Mocking isolates the unit under test by replacing real dependencies with fake objects (Mock obj)**.**

Concept && Coding

2. **Fast + no Flaky tests:**
   a. No I/O calls,
   b..No network calls.
   c.No db access

Tests become fast and stable.

3. **Controlled Output:**

Mocking allow us to control the behavior and response of dependencies.

Enable testing of edge cases (like timeout) and failure scenarios easily.

4. **Reduced tight coupling:**
   Without mocking, unit test depends on
   a.Database scheme
   b.Message broker
   c.External API etc..

Change in these system can cause unrelated unit tests to fail.

For ex:

DB schema got changed → results in failure of test cases which internally invokes DB.

<u>What's the difference between:</u> Stubbing, Mocking and Spying

<u>Stubbing</u>:

- Defines what a dependency should return.

```
@Test
void placeOrderSuccess() {

    PaymentService paymentService = mock(PaymentService.class);

   OrderService orderService = new OrderService(paymentService);
    /*
    This is stubbing.
    Here we are telling:

    If this method is invoked with this argument, then return this value
    or throw exception.
   */
    when(paymentService.pay(10)).thenReturn(true);

    boolean result = orderService.placeOrder(10);

    assertTrue(result);
}

```

- **Focus is only on input and output.**

Concept && Coding

<u>Mocking</u>:

- Stubbing is just one part of Mocking.

Mocking provides more features like:

- Was a method called?
- How many time?
- With what arguments?
- In what order?

For example:

```
@Test
void placeOrderSuccess() {

    PaymentService paymentService = mock(PaymentService.class);

    OrderService orderService = new OrderService(paymentService);

    //  Stubbing
    when(paymentService.pay(10)).thenReturn(true);

```

```
    boolean result = orderService.placeOrder(10);

    assertTrue(result);

    // Was the method called?
    verify(paymentService).pay(10);

    // How many times was it called?
    verify(paymentService, times(1)).pay(10);

    // With what arguments?
    verify(paymentService).pay(eq(10));

    // Was it the only interaction?
    verifyNoMoreInteractions(paymentService);
}

```

- **Focus is not only on input and output. But also on interaction (how system collaborates with its dependencies)**

Spying:

- Its partial mock of real object.
- Real method are invoked by default.
- But we can stub and verify selected method too.

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int multiply(int a, int b) {
        return a * b;
    }
}

```

Concept && Coding

```
Calculator calculator = spy(new Calculator());

// stubbing
when(calculator.multiply(4, 2)).thenReturn(100);

int sum = calculator.add(4, 2);       // invokes REAL method → 6
assertEquals(6, sum);

int product = calculator.multiply(4, 2);  // mocked → 100
assertEquals(100, product);


```

Output:

```
@Test
void testSpyWithWhen() {

    Calculator calculator = Mockito.spy(new Calculator());

    // stubbing
    Mockito.when(calculator.multiply( a: 4,  b: 2)).thenReturn( value: 100);

    int sum = calculator.add( a: 4,  b: 2);       // invokes REAL method → 6
    assertEquals( expected: 6, sum);

    int product = calculator.multiply( a: 4,  b: 2);  // mocked → 100
    assertEquals( expected: 100, product);
}
```

```
MokitoExample.testSpyWithWhen ×
MokitoExample                    200 ms     Tests passed: 1 of 1 test – 20
  testSpyWithWhen()              200 ms     /Users/shrayanshjain/Lib
```

- Need to be used carefully. There is a chance that we are trying to do stubbing but mistakenly also invoked real method.

```java
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int multiply(int a, int b) {
        return a * b;
    }
}

```

Concept && Coding

Like this, here stubbing is happening. But it will also invoke the multiply method.

```java

Calculator calculator = spy(new Calculator());

// stubbing: But also invokes real method
when(calculator.multiply(4, 2)).thenReturn(100);

int sum = calculator.add(4, 2);       // invokes REAL method → 6

int product = calculator.multiply(4, 2);  // mocked → 100

```

Safe implemenation, make use of : doReturn for stubbing, to avoid calling the real method.

```java
Calculator calculator = spy(new Calculator());

// stubbing: do not invoke real method, safe to use with spy
doReturn(100).when(calculator).multiply(4, 2);

int sum = calculator.add(4, 2);   // invokes REAL method → 6

int product = calculator.multiply(4, 2);   // mocked → 100

```

What? how come for When()...thenReturn() it is invoking the real method and why for doReturn().when() its not invoking the real method?

Concept && Coding